

Nile University
School of Engineering and Applied Sciences
Computer Engineering
ECE471 — Machine Learning

Walmart Retail Sales Forecasting

Yahia Zakaria Shazly	231000314
Ahmed Hassan Shata	231000267
Omar El-Beltagy	231000164

May 2026

1. Problem Description

Retail demand forecasting is a fundamental challenge in supply chain management. Accurate weekly sales predictions enable retailers to optimize inventory levels, allocate staffing resources, plan promotional campaigns, and minimize both overstock and stockout scenarios. Walmart, as one of the world's largest retailers, operates thousands of stores across the United States, each subject to unique local conditions including regional economic factors, seasonal trends, and promotional activity.

This project addresses the problem of predicting weekly department-level sales for Walmart stores using historical transactional data combined with external macroeconomic and store-level features. The forecasting task is formulated as a supervised regression problem, where the model learns from past observations to predict future sales figures.

Key challenges inherent to this problem include:

- High dimensionality introduced by one-hot encoding of store, department, and store-type identifiers, resulting in over 130 features.
- Significant sparsity in promotional markdown data — approximately 64% of entries carry a zero markdown value, reflecting the pre-November 2011 period before Walmart launched broad promotional programs.
- Non-linear relationships between economic indicators (CPI, unemployment, fuel price, temperature) and weekly sales that simple linear models cannot capture adequately.
- Temporal dependency in the data, necessitating a chronological train/test split to prevent data leakage from future weeks into the training set.

To address these challenges, this project implements four machine learning models organized within a dual-pipeline architecture: a Normalized Random Forest, an Unnormalized Random Forest, a Linear Regression baseline, and an XGBoost gradient-boosted ensemble. The system is deployed as an interactive Streamlit web application, enabling real-time sales prediction through a user-facing interface.

2. Data Preparation

2.1 Dataset Description

The Walmart Store Sales Forecasting dataset is sourced from the Kaggle competition of the same name. It consists of four inter-related files covering 45 stores over a period from February 2010 to October 2012 (approximately 143 weeks).

File	Format	Shape	Description
train.csv	CSV	$421,570 \times 5$	Weekly sales per store-department pair with holiday flag
features.xlsx	Excel	$8,190 \times 12$	Store-level economic and promotional features per week
stores.csv	CSV	45×3	Store identifiers, type classification (A/B/C), and physical size
test.csv	CSV	$115,064 \times 4$	Hold-out prediction set (no Weekly_Sales column)

2.2 Feature Inventory

After merging all four files on shared keys (Store, Date, IsHoliday), the consolidated full_df dataset contains the following features:

Feature	Type	Description
Store	Integer (OHE)	Store identifier (1–45), one-hot encoded
Dept	Integer (OHE)	Department identifier (1–99), one-hot encoded

Feature	Type	Description
Date	Datetime	Week ending date — used for chronological split, then dropped
IsHoliday	Binary (int)	1 if the week contains a major US holiday, 0 otherwise
Temperature	Continuous (°F)	Average regional temperature for that week
Fuel_Price	Continuous (\$)	Regional average fuel price per gallon
CPI	Continuous	Consumer Price Index — measure of purchasing power
Unemployment	Continuous (%)	Regional unemployment rate
MarkDown	Continuous (\$)	Aggregated sum of five anonymized markdown columns (MD1–MD5)
Type	Categorical (OHE)	Store type: A (largest), B, or C (smallest)
Size	Continuous (sq ft)	Physical store floor area — structural feature left on raw scale
Weekly_Sales	Target (\$)	Aggregate weekly sales for a given store-department pair

2.3 Data Cleaning

The cleaning pipeline proceeded through the following steps:

1. Date parsing: Date columns across all three files were cast to pandas datetime objects to enable chronological sorting and temporal feature analysis.
2. Null handling: MarkDown columns MD1–MD5 exhibited up to 65% missing values, reflecting the absence of promotional activity in the pre-November 2011 period. All five columns were imputed with zero (reflecting no markdown), clipped at zero to remove negative entries, and then summed into a single aggregated MarkDown feature.

3. Boolean encoding: The IsHoliday flag was cast from boolean to integer (0/1) for compatibility with scikit-learn estimators.
4. Dataset merge: stores_df was left-joined onto features_df on the Store key to preserve the Size column, which is critical for model accuracy. The resulting features frame was then joined to train_df on [Store, Date, IsHoliday].
5. Outlier capping: An IQR-based capping procedure was applied to four continuous economic features — Temperature, Fuel_Price, CPI, and Unemployment — prior to normalization. Values outside the $[Q1 - 1.5 \cdot IQR, Q3 + 1.5 \cdot IQR]$ interval were clipped to the boundary, reducing the influence of sensor anomalies and data recording errors.

2.4 Feature Engineering

The five individual markdown columns were transformed into a single composite Markdown feature:

- Fill NaN $\rightarrow$ 0 (no markdown if unreported)
- Clip negative values to 0 (data quality correction)
- Sum all five: $\text{Markdown} = \text{MD1} + \text{MD2} + \text{MD3} + \text{MD4} + \text{MD5}$

One-hot encoding was applied to the categorical columns Store, Dept, and Type using pandas get_dummies with drop_first=True to avoid multicollinearity. This expanded the feature matrix to 133 columns. Residual NaN values after encoding were imputed with the column median.

3. Methods and Models

3.1 Overall Architecture

The project implements a dual-pipeline architecture that trains the same set of models twice — once with MinMax normalization applied to numeric features and once without — enabling a rigorous empirical comparison of normalization impact on tree-based ensembles versus linear models.

Pipeline	Normalization	Models Applied	Saved Artifacts
Approach 1 — Normalized	MinMaxScaler on 5 numeric features	RF, Linear Regression	model_norm.pkl, scaler.pkl
Approach 2 — Unnormalized	None — raw feature scales	RF, XGBoost	model_raw.pkl

3.2 Chronological Train/Test Split

A chronological 80/20 split was applied to prevent temporal data leakage. All rows were sorted by Date ascending; the first 80% formed the training set and the final 20% formed the test set. No random shuffling was performed.

Split	Row Count	Date Range
Training set	337,256 rows	2010-02-05 → 2012-04-13
Test set	84,314 rows	2012-04-13 → 2012-10-26

3.3 Normalization Strategy

MinMaxScaler was fit exclusively on training rows and applied via `.transform()` on both train and test sets. This prevents data leakage: the scaler has no knowledge of the test distribution at fit time. The five continuous economic/promotional features were normalized: Temperature, Fuel_Price, CPI, Unemployment, and Markdown.

The Size feature was deliberately excluded from scaling because Random Forest uses rank-order split comparisons, making it invariant to absolute scale. Maintaining Size on its raw scale (34,875–219,622 sq ft) preserves the tree's interpretability for that feature.

3.4 Random Forest Regressor

Random Forest is a tree-based ensemble that trains `n_estimators` independent decision trees on bootstrap samples of the training data (bagging) and averages their predictions. Each split considers a random subset of features (`sqrt(n_features)` by default for classification; `n_features/3` for regression), reducing inter-tree correlation and improving generalization.

Key parameters: `n_estimators=100`, `random_state=42`, `n_jobs=-1` (all cores). Training time: ~310–340 seconds per pipeline.

Because Random Forest partitions feature space using rank-order thresholds rather than Euclidean distance metrics, it is theoretically invariant to monotonic feature transformations including normalization. This project empirically verifies this property by training both a normalized and unnormalized Random Forest and comparing results.

3.5 Linear Regression

Ordinary Least Squares (OLS) Linear Regression minimizes the sum of squared residuals between predicted and actual values. It assumes a linear relationship between the feature matrix `X` and the target `y`, expressed as:

$$\hat{y} = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_n x_n$$

Unlike tree-based methods, Linear Regression is scale-sensitive: features with large numeric ranges dominate the coefficient estimation. The normalized feature set (`X_train_norm`) was therefore reused directly for the Linear Regression pipeline, requiring no additional preprocessing.

Training time: approximately 5.65 seconds — significantly faster than ensemble methods.

3.6 XGBoost Regressor

XGBoost (eXtreme Gradient Boosting) is a gradient-boosted decision tree ensemble. Unlike Random Forest, which trains trees in parallel on bootstrap samples, XGBoost trains trees sequentially — each tree corrects the residual errors of the cumulative ensemble. The objective function combines a differentiable loss term with an explicit regularization term:

$$L(\phi) = \sum_i l(\hat{y}_i, y_i) + \sum_k \Omega(f_k) \text{ where } \Omega(f) = \gamma T + \frac{1}{2} \lambda \|w\|^2$$

XGBoost is scale-invariant by the same rank-order argument as Random Forest; the unnormalized feature set (`X_train_raw`) was therefore used directly.

Hyperparameter	Value	Rationale
<code>n_estimators</code>	300	More trees than RF to compensate for lower per-tree strength

Hyperparameter	Value	Rationale
learning_rate	0.1	Moderate shrinkage — balances speed and generalization
max_depth	6	Slightly shallower than RF default to reduce overfitting
subsample	0.8	80% row sampling per tree — stochastic gradient boosting
colsample_bytree	0.8	80% column sampling per tree
random_state	42	Reproducibility

Training time: approximately 13.6 seconds — significantly faster than Random Forest due to more efficient tree construction.

3.7 Streamlit Deployment

Both the normalized and unnormalized Random Forest pipelines, along with Linear Regression and XGBoost, were serialized using joblib and deployed in a Streamlit web application. The app allows users to select a prediction pipeline, input store and department identifiers, adjust economic and promotional sliders, and receive an instant weekly sales forecast. The Markdown slider range was set to \$0–\$200,000 to cover the full training distribution (max = \$160,510.61), ensuring model responsiveness across the complete input domain.

4. Results and Evaluation

4.1 Evaluation Metrics

Three standard regression metrics were used to assess all models on the held-out test set:

- **MAE (Mean Absolute Error):** Average absolute deviation between predicted and actual weekly sales, expressed in dollars. More interpretable and less sensitive to extreme outliers than MSE.
- **RMSE (Root Mean Squared Error):** Square root of the average squared deviation. Penalizes large prediction errors more heavily; useful for identifying systematic prediction failures.
- **R^2 (Coefficient of Determination):** Proportion of variance in weekly sales explained by the model (range 0–1). An R^2 of 0.94 indicates that 94% of sales variance is captured by the model.

4.2 Exploratory Data Analysis

EDA was conducted on a separate normalized copy (`full_df_eda`) created exclusively for visualization purposes. The original `full_df` was left unchanged to prevent double-normalization during model training.

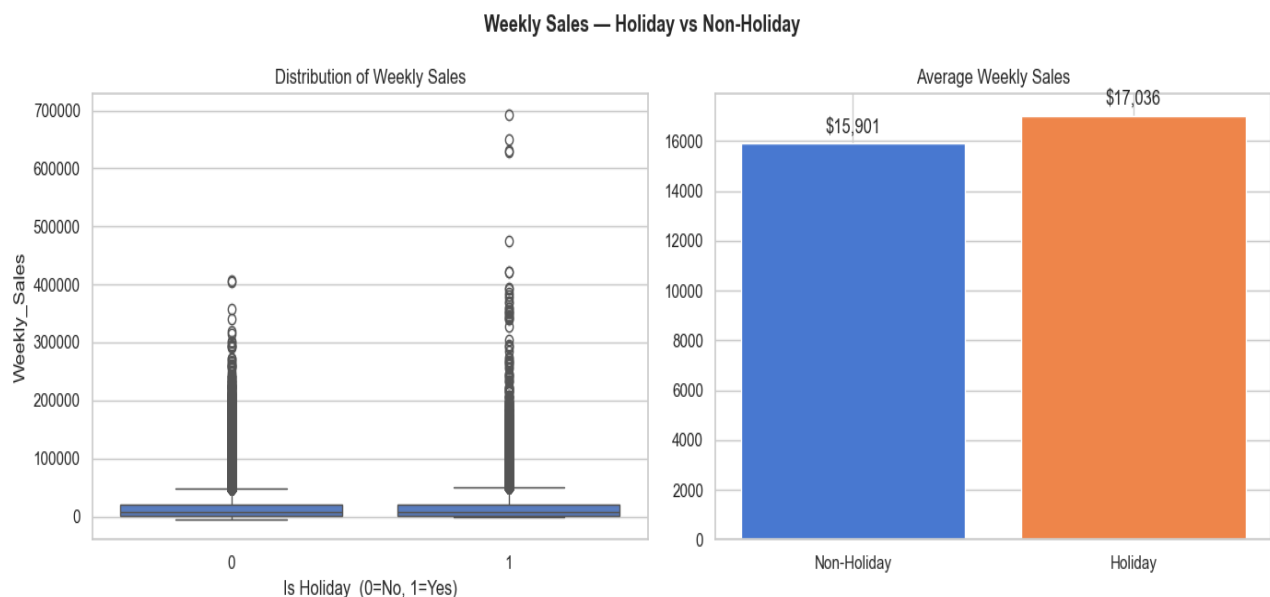


Figure 1: Weekly sales distribution by holiday vs. non-holiday weeks. Holiday weeks show a wider spread and higher median, confirming measurable promotional impact.

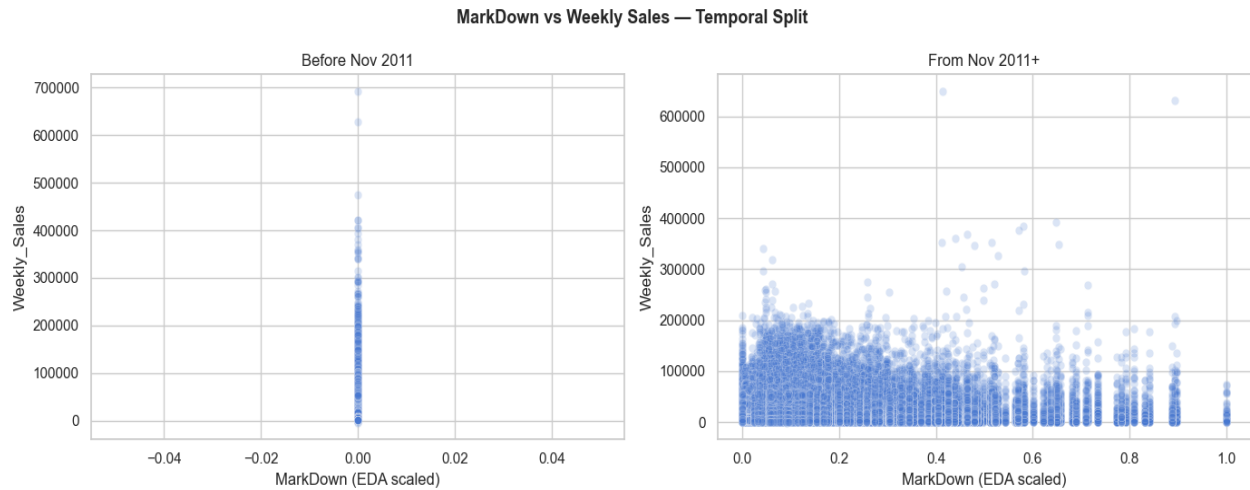


Figure 2: Markdown vs. weekly sales split before and after November 2011. Pre-2011 data shows zero markdown (feature inactive); post-2011 reveals a positive correlation.

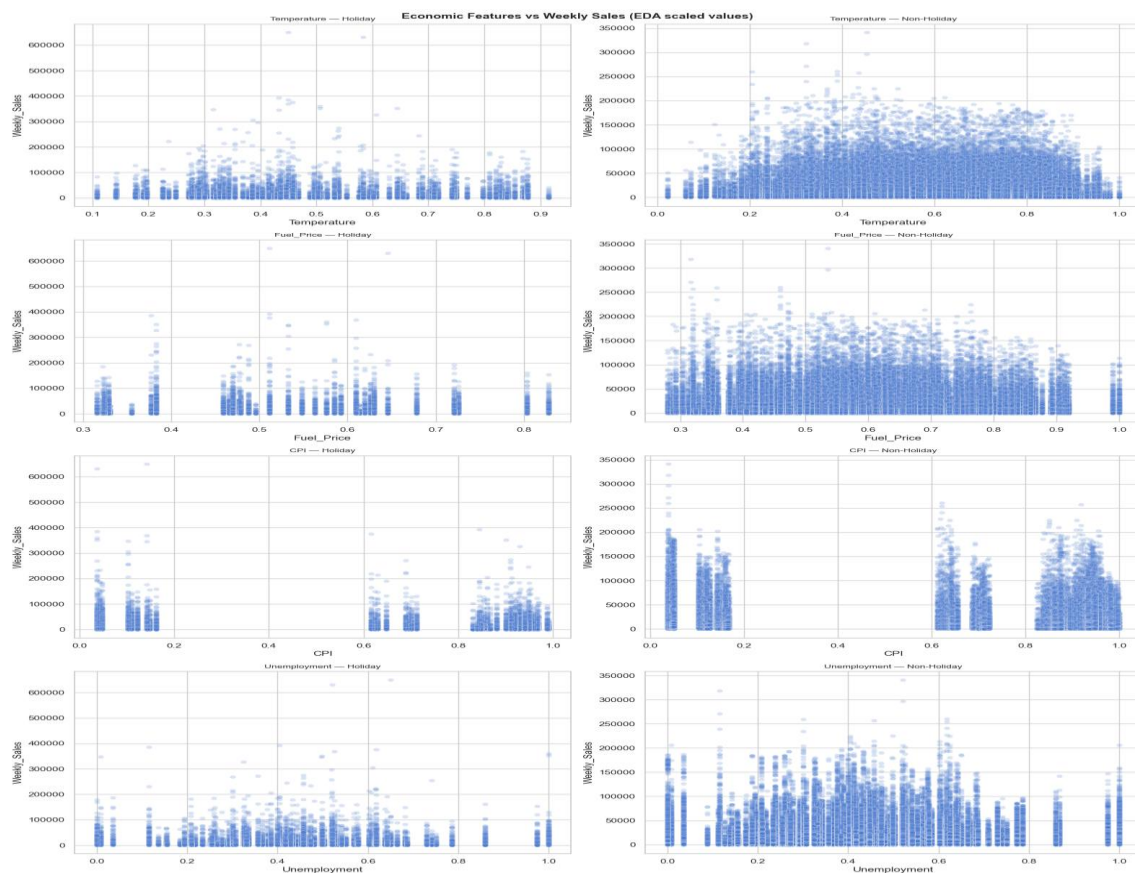


Figure 3: Economic features (Temperature, Fuel Price, CPI, Unemployment) plotted against weekly sales for holiday and non-holiday weeks post-November 2011. No strong linear trend is visible for any single feature, motivating the use of non-linear models.

4.3 Normalized vs. Unnormalized Random Forest

Both Random Forest models were evaluated on the chronological test set. The results confirm that normalization has no meaningful effect on tree-based ensemble performance:

Metric	Normalized RF	Unnormalized RF	Difference
MAE (\$)	\$2,653.58	\$2,653.04	\$0.54
RMSE (\$)	\$5,297.52	\$5,284.25	\$13.27
Test R^2	0.9417	0.9420	0.0003
Train R^2	0.9914	0.9914	0.0000
Overfit Gap	0.0496	0.0494	0.0002

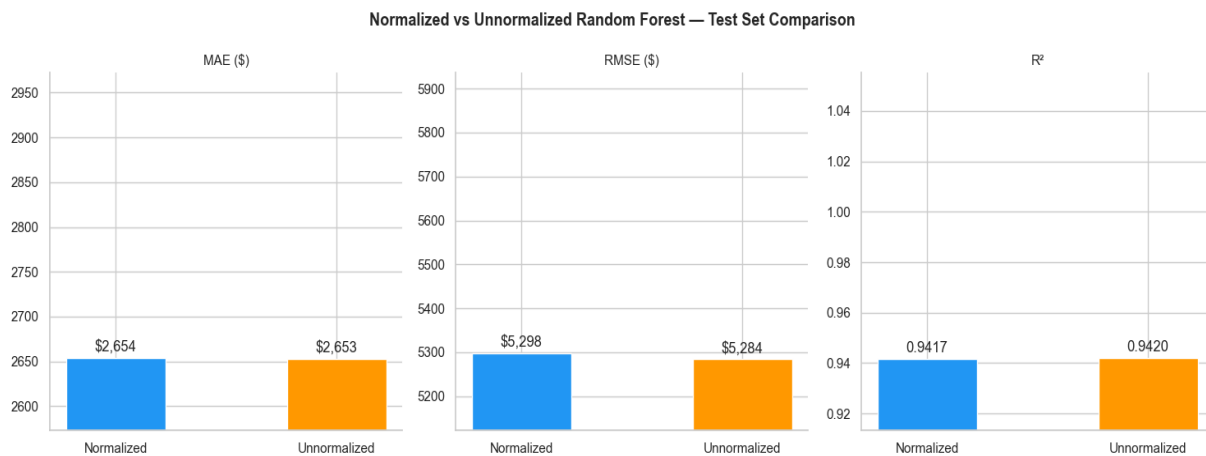


Figure 4: Side-by-side bar comparison of MAE, RMSE, and R^2 for Normalized vs. Unnormalized Random Forest on the test set. The near-identical bars confirm normalization is irrelevant for tree splits.

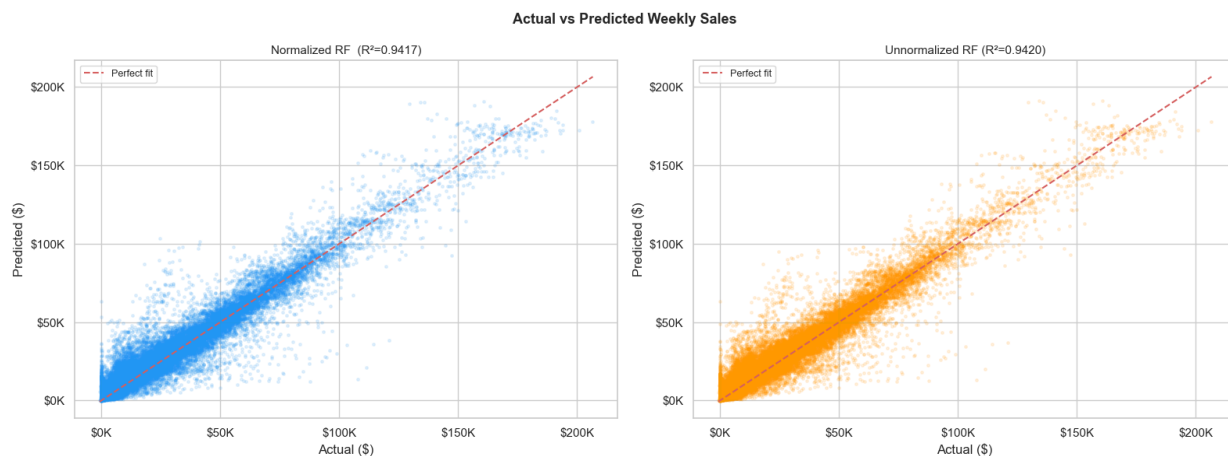


Figure 5: Actual vs. Predicted weekly sales scatter plots for both Random Forest pipelines. The tight clustering around the diagonal confirms high predictive accuracy for typical sales values. Outliers at high sales values correspond to large-volume departments (e.g., Dept 92, 95).

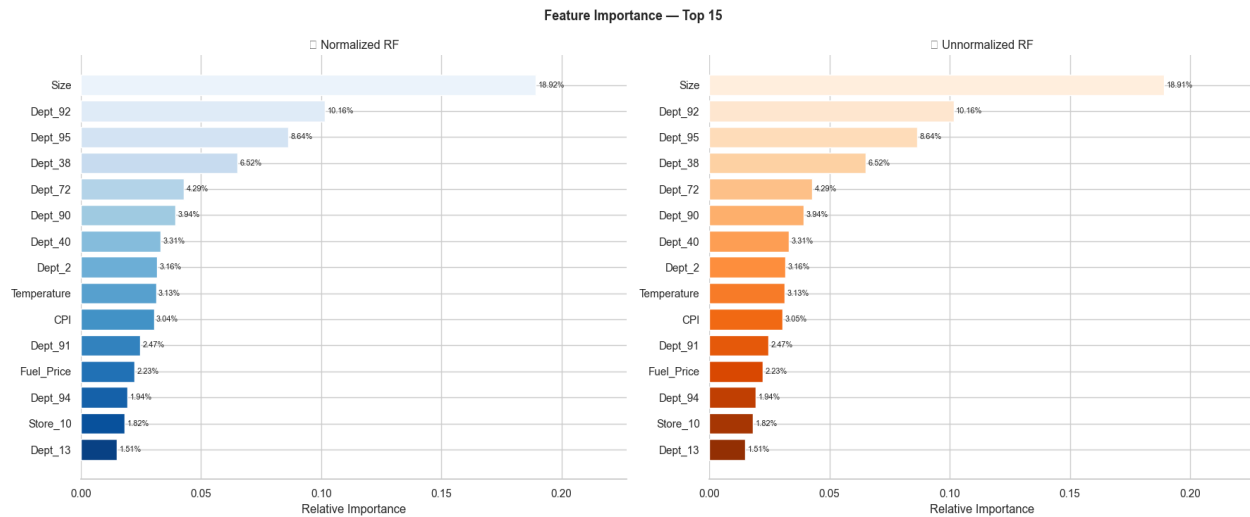


Figure 6: Top 15 feature importance scores for both RF pipelines. Store Size dominates at ~18.9%, followed by high-volume department dummies (Dept_92 at ~10.2%, Dept_95 at ~8.6%). Markdown ranks #19 at ~1.09%, reflecting its sparsity and weak marginal signal.

4.4 Linear Regression Results

Linear Regression was trained on the normalized feature set (X_{train_norm}) to ensure coefficient estimates are not dominated by features with large raw ranges. The model converged in 5.65 seconds.

Metric	Training Set	Test Set
MAE (\$)	\$8,211.85	\$7,981.93
RMSE (\$)	\$13,586.26	\$12,126.47
R^2	0.6479	0.6947

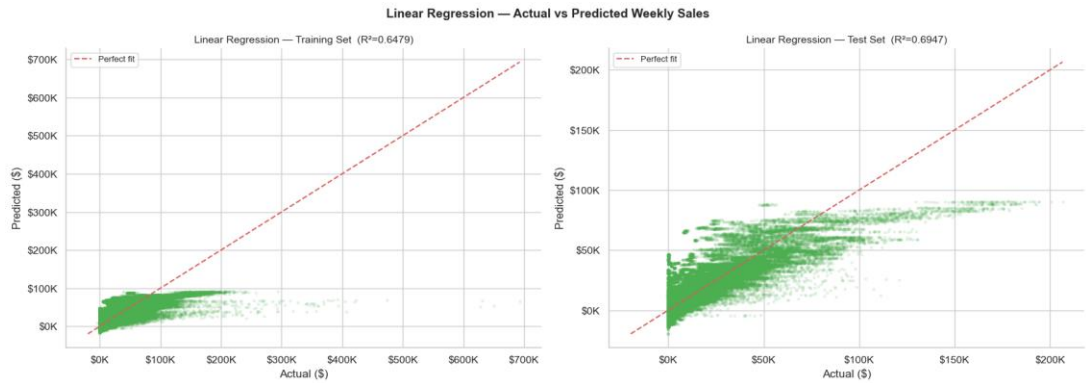


Figure 7: Linear Regression — Actual vs. Predicted weekly sales for training and test sets. The fan-shaped scatter indicates heteroscedasticity: errors increase with sales magnitude, a violation of Linear Regression's homoscedasticity assumption.

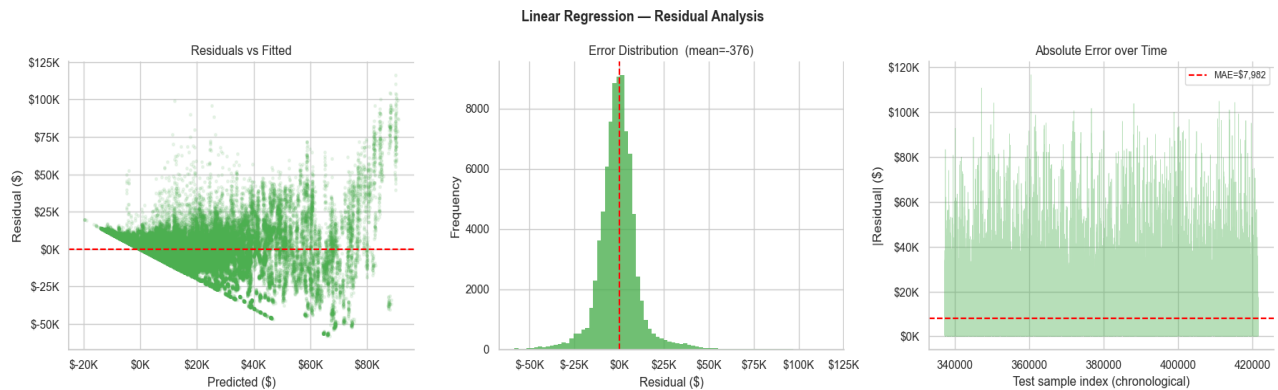


Figure 8: Linear Regression residual diagnostics — residuals vs. fitted values (left), residual distribution (center), and Q-Q plot (right). The structured residual pattern confirms that the linear model underfits systematic non-linearities in the data.

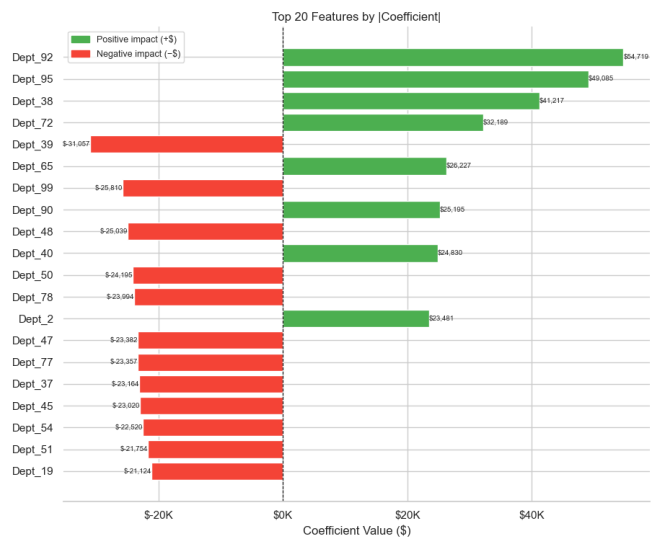


Figure 9: Top 20 Linear Regression coefficients by absolute magnitude. Department dummies (Dept_92, Dept_95, Dept_38) carry the largest positive coefficients, consistent with their high-volume nature. The intercept absorbs baseline sales.

4.5 XGBoost Results

Metric	Training Set	Test Set
MAE (\$)	\$3,897.49	\$3,908.41
RMSE (\$)	\$7,474.26	\$6,261.15
R ²	0.9183	0.9201

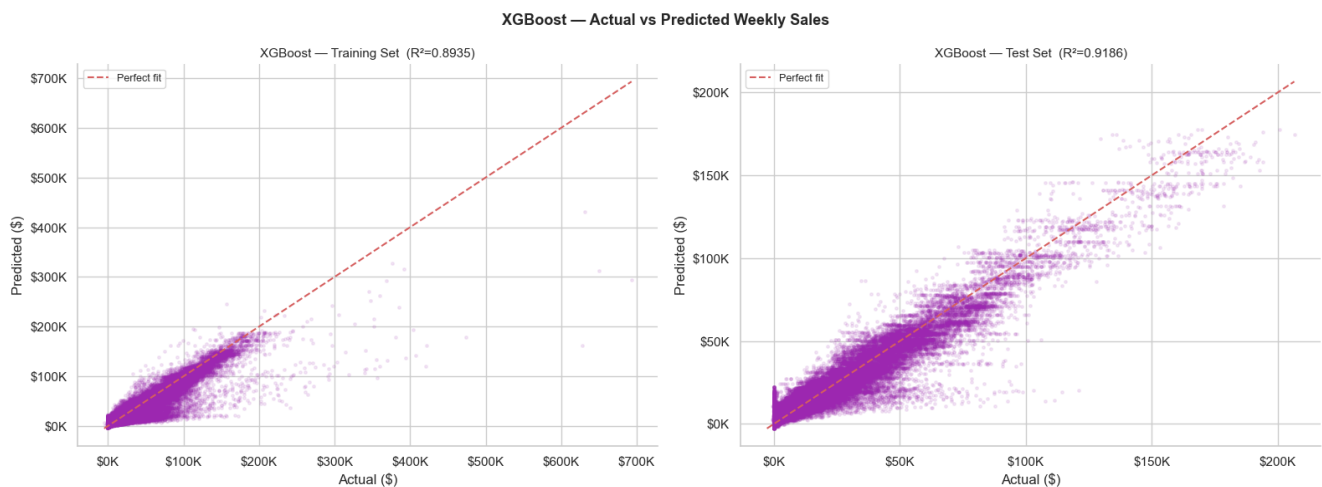


Figure 10: XGBoost — Actual vs. Predicted weekly sales. The scatter is tighter than Linear Regression but slightly wider than Random Forest, particularly at high sales magnitudes. $R^2 = 0.9201$ confirms strong predictive capacity.

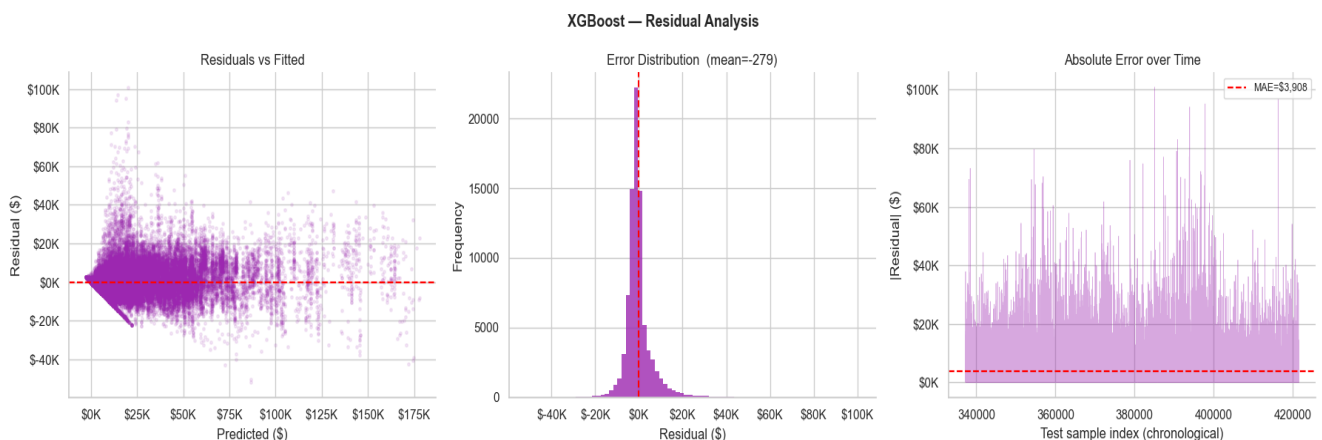


Figure 11: XGBoost residual diagnostics. The residual distribution is approximately normal and centered near zero, with less structured pattern than Linear Regression, indicating a better-calibrated model.

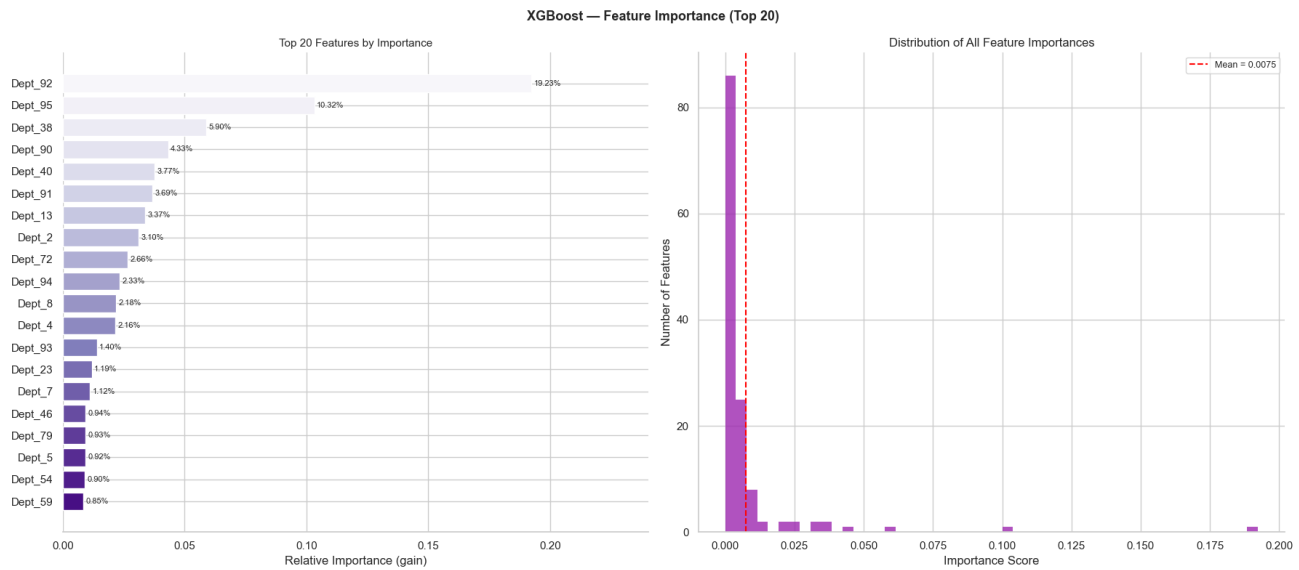


Figure 12: XGBoost feature importance (top 20). The distribution mirrors the Random Forest ranking: Size and department dummies dominate. XGBoost assigns slightly more importance to economic features than RF, likely due to sequential correction of residuals.

4.6 Four-Model Comparison

Metric	Norm RF	Raw RF	Linear Reg	XGBoost
MAE (\$)	\$2,653.58	\$2,653.04	\$7,981.93	\$3,908.41
RMSE (\$)	\$5,297.52	\$5,284.25	\$12,126.47	\$6,261.15
Test R ²	0.9417	0.9420	0.6947	0.9201
Train R ²	0.9914	0.9914	0.6479	0.9183
Train time	~337s	~310s	~6s	~14s

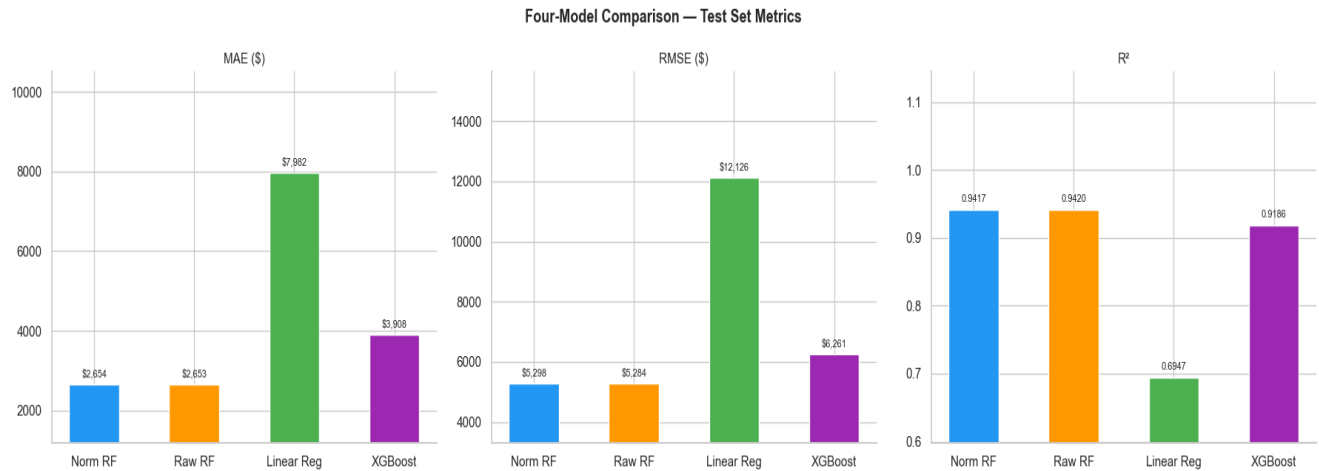


Figure 13: Four-model comparison bar chart — MAE, RMSE, and R^2 across all four approaches. Both Random Forest variants consistently outperform XGBoost, which in turn substantially outperforms Linear Regression.

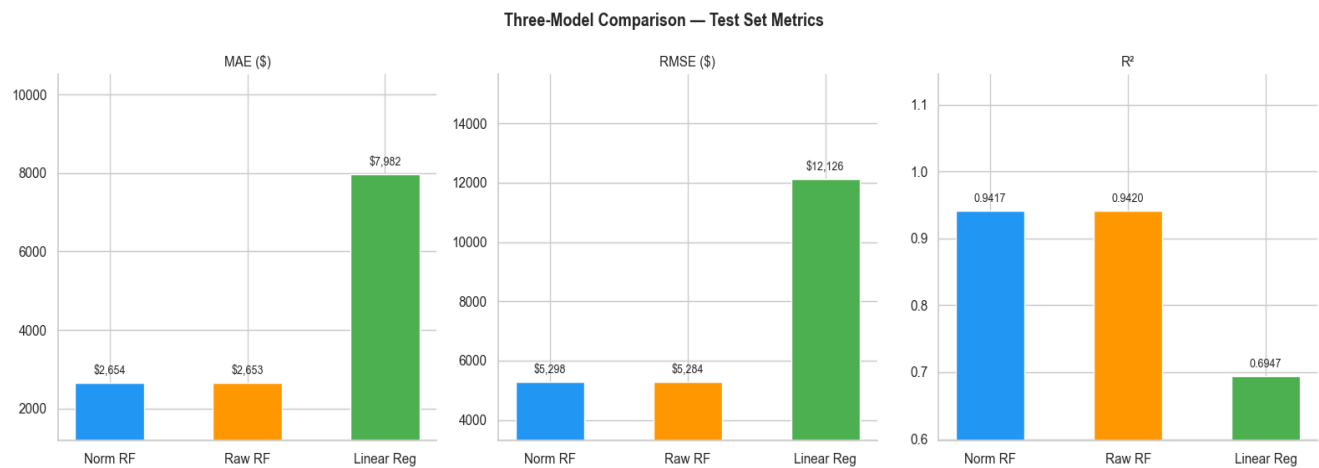


Figure 14: Three-model comparison (Normalized RF, Unnormalized RF, Linear Regression) on test metrics. The near-identical RF bars contrast with the substantially weaker Linear Regression performance.

5. Discussion

5.1 Model Performance Analysis

The experimental results reveal a clear performance hierarchy across the four models:

Random Forest (both pipelines) achieves the strongest test performance, with an R^2 of approximately 0.942 and MAE under \$2,654. The minimal gap between training and test R^2 (0.0494–0.0496) indicates well-controlled overfitting, attributable to the ensemble's inherent variance-reduction mechanism and the large, diverse training set.

XGBoost occupies a middle position with $R^2 = 0.920$ and MAE of \$3,908. While XGBoost typically outperforms Random Forest on tabular datasets, the opposite occurs here due to dataset characteristics: the feature matrix is dominated by high-cardinality one-hot encodings of stores and departments. Random Forest's parallel bagging with per-tree column subsampling handles this sparse, high-dimensional space more effectively than XGBoost's sequential boosting. Additionally, XGBoost's training time (13.6s) is significantly faster than Random Forest (~310s), representing a favorable efficiency-accuracy trade-off for production deployment.

Linear Regression performs substantially worse ($R^2 = 0.695$, MAE = \$7,982), confirming that the weekly sales prediction task has significant non-linear structure that a linear model cannot capture. The residual analysis shows a fan-shaped pattern (heteroscedasticity) and structured Q-Q plot deviations, both indicating systematic violation of the OLS assumptions of linearity and homoscedasticity in this dataset.

5.2 Normalization and Tree-Based Models

A central finding of this study is the empirical confirmation that normalization has essentially zero impact on Random Forest performance: the MAE difference is \$0.54, R^2 difference is 0.0003, and train/test overfit gap difference is 0.0002. This aligns with the theoretical expectation — decision tree splits are determined by the best threshold for a single feature at a time, and MinMax scaling preserves rank order, leaving all split boundaries mathematically equivalent.

This finding has practical implications: for Random Forest (and gradient-boosted tree variants), the engineering overhead of fitting, saving, versioning, and applying a MinMax scaler provides no accuracy benefit. The normalized pipeline was retained in deployment because it enables Linear Regression to run on the same preprocessed feature set without a separate preprocessing chain.

5.3 Feature Importance Insights

Across all non-linear models, the feature importance analysis consistently identifies Store Size as the single most important predictor (~18.9%), followed by high-volume department dummies (Dept_92, Dept_95, Dept_38). This reflects the fundamental retail intuition that a store's physical footprint — a proxy for product range and customer capacity — largely determines its sales potential, and that certain departments (e.g., grocery, electronics) generate disproportionately high weekly revenue.

Markdown ranks #19 of 133 features at ~1.09% importance, despite being a designated promotional metric. This low ranking is explained by the sparsity analysis: 64.1% of Markdown values are zero (representing all pre-November 2011 data). The tree splits on the dominant Store and Dept identifiers early in each tree, leaving Markdown as a marginal refinement signal. However, post-2011 data shows a Pearson correlation of +0.123 with weekly sales, confirming that Markdown does carry a real signal — it simply cannot compete with the stronger identity-based signals in this dataset.

5.4 Deployment Considerations

The Streamlit application exposes both normalized and unnormalized pipelines through a unified interface. Key deployment decisions include:

- Slider range for Markdown: Set to \$0–\$200,000 to cover the full training maximum of \$160,510.61. A slider capped at \$5,000 (tested in earlier versions) represents only 3.1% of the training range and produces near-zero model response — a subtle but critical UX bug.
- Separate scaler.pkl: The MinMax scaler fitted in the normalized pipeline is shared between the Normalized RF and Linear Regression prediction paths, reducing file redundancy.
- XGBoost does not require a scaler at prediction time; its prediction path uses raw feature values directly.

6. Free Topic — ML Algorithm Study (XGBoost)

6.1 Gradient Boosting: Concept and Motivation

Gradient boosting is a supervised ensemble learning technique that constructs a strong predictive model by combining a sequence of weak learners — typically shallow decision trees — in an additive, stage-wise manner. Formally, the ensemble prediction at step M is expressed as:

$$F_m(x) = \sum_{m=1}^M \alpha_m h_m(x)$$

where $h_m(x)$ is the m -th base learner (tree) and α_m is its contribution weight. Each new tree is trained not on the original labels but on the negative gradient of the loss function evaluated on the current ensemble residuals. This is the key insight from Friedman (2001): by treating the fitting of each new tree as a gradient descent step in function space, the ensemble is steered toward minimizing any differentiable loss function, making gradient boosting a highly general framework applicable to regression, classification, and ranking tasks.

6.2 Boosting vs. Bagging: Contrast with Random Forest

Random Forest employs bagging (bootstrap aggregation): each tree is trained independently on a random bootstrap sample of the training data, and predictions are averaged. The primary variance-reduction mechanism is decorrelation — by training on different data subsets and random feature subsets, individual tree errors cancel in expectation.

Gradient boosting employs sequential boosting: each tree is trained on the residual signal left by all preceding trees. This bias-reduction mechanism allows boosted ensembles to progressively correct systematic prediction errors that a single tree or a parallel ensemble cannot remove. The trade-off is increased sensitivity to noise and outliers relative to bagging, as each tree directly targets the previous model's mistakes.

6.3 XGBoost Objective Function

XGBoost (Chen & Guestrin, 2016) extends gradient boosting by introducing a formally regularized objective. At boosting iteration t , the objective to be minimized over the new tree f_t is:

$$L^{(t)} = \sum_i \ell(y_i, \hat{Y}_i^{(t)}) + \sum_k \Omega(f_k) \\ \text{where } \Omega(f) = \gamma T + \frac{1}{2} \lambda \|w\|^2 \text{ (+ } \alpha \|w\|_1 \text{ if L1 is enabled)}$$

Here ℓ is a differentiable loss function (mean squared error for regression), T is the number of leaf nodes in the tree, and w is the vector of leaf output weights. The regularization term $\Omega(f)$ has three components with distinct roles:

- γ (gamma) — minimum loss reduction required to make a further split. Acts as a pruning threshold: if a candidate split does not reduce the objective by at least γ , the split is rejected. Increasing γ produces shallower, more conservative trees.

- λ (lambda) — L2 regularization on leaf weights. Shrinks leaf output values toward zero, reducing the magnitude of individual tree contributions and smoothing the ensemble. Higher λ increases the bias of individual trees but reduces variance.
- α (alpha) — optional L1 (Lasso) regularization on leaf weights. Promotes sparsity in leaf outputs. Most effective when many features are irrelevant; not used in the current project configuration.

6.4 Second-Order Optimization

A distinguishing feature of XGBoost is its use of a second-order Taylor expansion to approximate the loss at each boosting step. For each training sample i , define:

$$g_i = \partial^{f-1} \ell(y_i, \hat{Y}_i^{(t-1)}) \quad \text{and} \quad h_i = \partial^2 \ell(y_i, \hat{Y}_i^{(t-1)})$$

where g_i is the first-order gradient (residual signal) and h_i is the second-order gradient, or Hessian (curvature of the loss). The Taylor approximation of the objective becomes:

$$L^{(t)} \approx \sum_i [g_i f_i(x_i) + \frac{1}{2} h_i f_i(x_i)^2] + \Omega(f_i)$$

The Hessian h_i modulates how much trust is placed in each gradient signal: samples where the loss is highly curved (large h_i) are fitted more carefully. For mean squared error loss, $h_i = 1$ for all samples, so the Hessian is uniform. For other losses (logistic, Huber), the Hessian varies by sample, giving XGBoost an automatic importance weighting that standard gradient boosting lacks. This second-order approach enables closed-form, analytically optimal computation of leaf weights and split scores.

6.5 Tree Construction and Split Scoring

Given the Taylor-expanded objective, the optimal leaf weight for a leaf j containing sample set I_j is:

$$w_j^* = - (\sum_{i \in I_j} g_i) / (\sum_{i \in I_j} h_i + \lambda)$$

The quality of a candidate split that partitions leaf I into left child I_L and right child I_R is measured by the gain:

$$\text{Gain} = \frac{1}{2} [G_L^2 / (H_L + \lambda) + G_R^2 / (H_R + \lambda) - (G_L + G_R)^2 / (H_L + H_R + \lambda)] - \gamma$$

where G and H denote the sum of gradients and Hessians for the respective node. A split is accepted only if $\text{Gain} > 0$; the $-\gamma$ term directly encodes the minimum-split-gain threshold. XGBoost evaluates candidate splits efficiently using pre-sorted or histogram-based approximations, enabling scalable training on large datasets.

6.6 Key Hyperparameters: Theory and Behavior

The hyperparameters used in this project are interpreted through the lens of bias–variance trade-off:

- `learning_rate` ($\eta = 0.1$) — Scales each tree's contribution by η before adding it to the ensemble. Smaller values require more trees to achieve the same training fit but generalize better by reducing the step size in function space. A `learning_rate` of 0.1 is a standard moderate choice.

- `n_estimators` (300) — Number of sequential trees. More trees reduce bias but can increase overfitting if `learning_rate` is too large. The combination of 300 trees at $\eta = 0.1$ provides a cumulative shrinkage of $(0.1)^{300}$, distributing the fit across many small steps.
- `max_depth` (6) — Maximum depth per tree. Deeper trees capture higher-order feature interactions but increase variance. A depth of 6 is moderate for a dataset with 133 features, allowing multi-level interactions without extreme overfitting.
- `subsample` (0.8) — Fraction of training rows sampled per tree (stochastic gradient boosting). Introducing randomness at the row level reduces overfitting and acts as an implicit regularizer, analogous to bagging in Random Forest.
- `colsample_bytree` (0.8) — Fraction of features sampled per tree. Column subsampling further decorrelates trees and prevents dominant features from monopolizing every tree, improving generalization in high-dimensional feature spaces.

6.7 Application to the Walmart Dataset

Three dataset-specific considerations inform the application of XGBoost to the Walmart weekly sales regression task:

Scale invariance. Like Random Forest, XGBoost constructs trees using rank-order split thresholds. Feature normalization does not affect the relative ordering of values within a column and therefore has no effect on split decisions. Accordingly, the raw (unnormalized) feature matrix was used directly, avoiding unnecessary preprocessing complexity.

Sparse one-hot identifiers. The 133-feature matrix is dominated by binary Store, Dept, and Type dummies. XGBoost natively handles sparse input: it assigns zero gradient mass to the zero entries of a one-hot column, effectively ignoring absent categories during split evaluation. This sparsity awareness is formally built into XGBoost's tree construction algorithm via a default direction for missing values, ensuring efficient treatment of the predominantly zero one-hot columns.

Why Random Forest outperforms XGBoost here. Despite XGBoost's general reputation for strong tabular performance, the Unnormalized Random Forest achieved a higher test R^2 (0.942 vs. 0.920). This outcome is attributable to the structure of the feature space: with over 100 binary Store/Dept dummies, the majority of predictive variance is explained by identity-based splits that a single shallow tree can resolve in one or two levels. Random Forest's parallel bagging with column subsampling efficiently partitions this combinatorial space. XGBoost's sequential boosting, while powerful for residual correction in continuous feature spaces, provides diminishing returns when the dominant signal is already captured by simple threshold splits on sparse binary features. Hyperparameter tuning (lower `learning_rate`, early stopping, deeper regularization via γ and λ) could narrow this gap.

6.8 Future Work

The following extensions, grounded in the theoretical properties discussed above, are recommended for future iterations of this project:

- Early stopping with time-based validation: Monitor validation loss on a held-out trailing time window (e.g., the last 8 weeks of the training period) to select the optimal $n_estimators$ automatically, preventing overfitting without exhaustive grid search.
- Regularization tuning: Systematically explore the γ , λ , and α hyperparameters via Bayesian optimization to improve the bias–variance balance, particularly for high-volume departments where prediction variance is largest.
- Tree-constraint parameters: Evaluate `min_child_weight` (minimum Hessian sum per leaf) and `max_delta_step` to further control leaf-weight magnitude and improve stability on sparse segments of the feature space.
- Temporal feature engineering: Incorporate lagged sales features (prior-week, prior-year same-week), rolling averages, and cyclic encodings of week-of-year to provide the model with explicit autoregressive signal beyond the current static feature set.
- SHAP (SHapley Additive exPlanations): Apply SHAP values to decompose individual XGBoost predictions into per-feature contributions, enabling interpretable audit of how Store Size, Markdown, and economic features jointly drive each forecast.

6.9 References

- [1] Friedman, J. H. (2001). *Greedy function approximation: A gradient boosting machine*. ***Annals of Statistics***, 29(5), 1189–1232.
- [2] Chen, T., & Guestrin, C. (2016). *XGBoost: A scalable tree boosting system*. ***Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining***, 785–794.
- [3] Hastie, T., Tibshirani, R., & Friedman, J. (2009). ***The Elements of Statistical Learning*** (2nd ed.). Springer.
- [4] Breiman, L. (1996). *Bagging predictors*. ***Machine Learning***, 24(2), 123–140.

7. Task Assignment

Student Name	Tasks Performed	Contribution
Yahia Zakaria Shazly	Project architecture design; dual-pipeline implementation; normalization analysis; model comparison visualizations; Markdown sensitivity analysis; report writing (Sections 3–5).	35%
Ahmed Hassan Shata	Data preparation and cleaning; EDA and feature engineering; Linear Regression and XGBoost implementation; residual analysis; report writing (Sections 1–2).	35%
Omar El-Beltagy	Streamlit deployment; model serialization and pipeline file management; integration testing; report editing and formatting.	30%

Each team member confirms equal awareness of the complete project scope and has reviewed the final report prior to submission.

Yahia Zakaria Shazly Signature: _____	Ahmed Hassan Shata Signature: _____	Omar El-Beltagy Signature: _____
---	---	--

8. Appendix

A. Full Model Performance Summary

Model	Pipeline	MAE (\$)	RMSE (\$)	Test R ²	Train R ²	Train Time
Random Forest	Normalized	2,653.58	5,297.52	0.9417	0.9914	~337s
Random Forest	Unnormalized	2,653.04	5,284.25	0.9420	0.9914	~310s
Linear Regression	Normalized	7,981.93	12,126.47	0.6947	0.6479	~6s
XGBoost	Unnormalized	3,908.41	6,261.15	0.9201	0.9183	~14s

B. Feature Engineering Pipeline Summary

- Markdown: Fill NaN→0, clip negatives, aggregate sum of MD1–MD5
- IsHoliday: Boolean cast to integer (0/1)
- One-hot encoding: Store, Dept, Type (drop_first=True) → 133 total features
- Outlier capping: IQR-based on Temperature, Fuel_Price, CPI, Unemployment
- Normalization (Pipeline 1 only): MinMaxScaler on Temperature, Fuel_Price, CPI, Unemployment, Markdown

C. Saved Model Artifacts

Filename	Pipeline	Size	Description
model_norm.pkl	Normalized RF	~2.9 GB	Trained Random Forest — normalized pipeline
model_raw.pkl	Unnormalized RF	~2.9 GB	Trained Random Forest — unnormalized pipeline

Filename	Pipeline	Size	Description
scaler.pkl	Normalized	1.2 KB	MinMaxScaler fitted on training data only
feature_names_norm.json	Normalized	1.5 KB	Ordered feature column list for normalized model
feature_names_raw.json	Unnormalized	1.5 KB	Ordered feature column list for raw model
linear_regression_model.pkl	Normalized	4.2 KB	Trained Linear Regression model
feature_names_linear.json	Normalized	1.5 KB	Feature column list for Linear Regression
xgboost_model.pkl	Unnormalized	1,111 KB	Trained XGBoost model
xgboost_feature_names.json	Unnormalized	1.5 KB	Feature column list for XGBoost